Deep Learning Fundamentals Made Easy

BSIT Data Science Lecture Handout

Concepts • Neural Networks • Architectures • Practical Examples

Contents

1 Introduction to Deep Learning

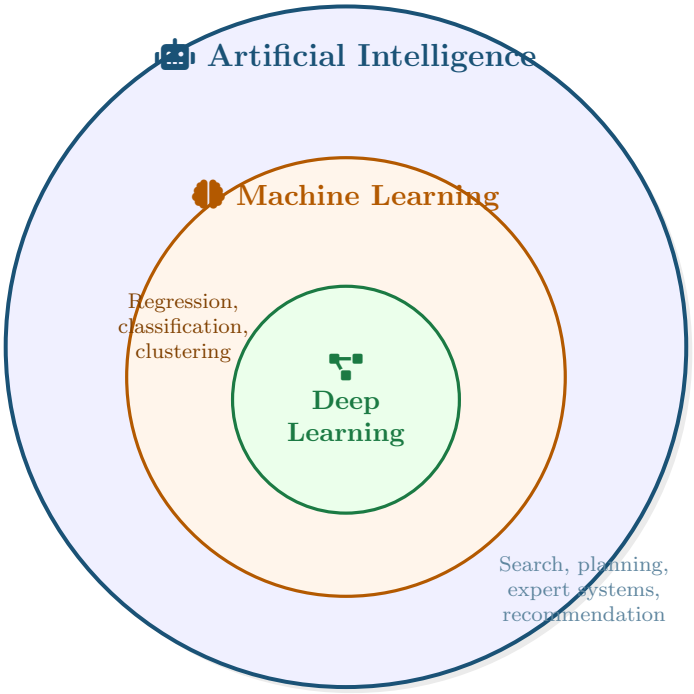
Deep Learning

Deep learning is a branch of machine learning that uses **artificial neural networks with multiple layers** to learn patterns directly from data. It is especially powerful for complex data such as **images, text, speech, video, and large tabular datasets**.

Simple Idea

Think of deep learning as **stacking many small pattern detectors**. Early layers may learn simple patterns such as edges, colors, or short word sequences. Deeper layers combine those simple patterns into richer ideas such as objects in an image, sentiment in a sentence, or fraud patterns in transactions.

1.1 Where Deep Learning Fits in AI



- **Artificial Intelligence (AI)** is the broad field of making machines behave intelligently.
- **Machine Learning (ML)** is a subset of AI that learns patterns from data.
- **Deep Learning (DL)** is a subset of ML that uses many-layer neural networks.

1.2 Why Deep Learning Matters

- It can learn useful features automatically instead of depending only on manual feature engineering.
- It performs very well when the data is large and complex.
- It powers modern systems such as face recognition, chatbots, translation, speech assistants, recommendation engines, and medical image analysis.
- It supports both predictive tasks and generative tasks such as text generation or image generation.

1.3 When Deep Learning Is a Good Choice

Deep Learning Often Fits Well	Traditional ML May Be Better First
Large image, audio, or text datasets	Small datasets with limited samples
Very complex relationships in the data	Problems where high interpretability is required
Tasks where automatic feature learning helps	Cases where a simple model already works well
Applications that can use GPU acceleration	Projects with very limited computing resources

Important Note

Deep learning is not always the best first model. In many business and classroom projects, a simpler method such as logistic regression, decision trees, or random forests may be easier to train, explain, and deploy.

2 Neural Network Basics

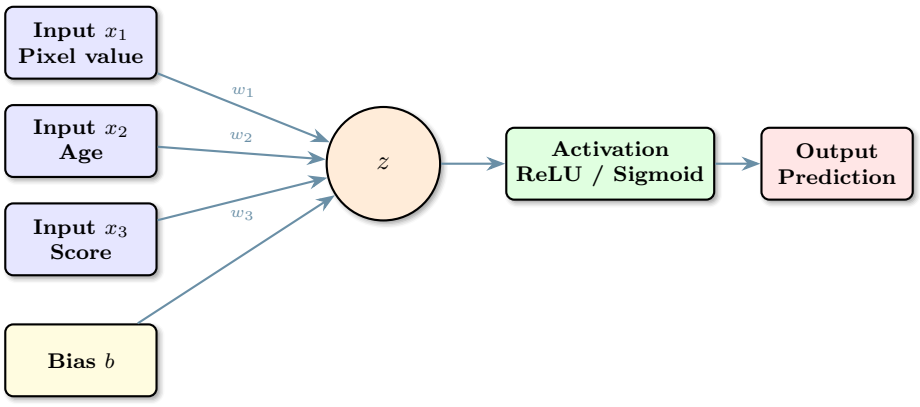
Artificial Neuron

An artificial neuron receives inputs, gives each input a **weight**, adds a **bias**, and then applies an **activation function** to produce an output.

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b \qquad a = \text{activation}(z)$$

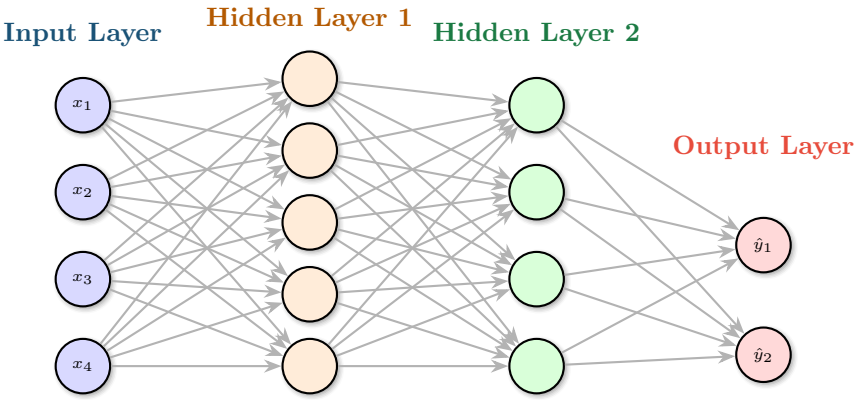
This is one of the few formulas you need to remember. Everything else in deep learning is built by connecting many of these simple units together.

2.1 Inside One Neuron



- **Inputs** are the feature values.
- **Weights** tell the model how important each input is.
- **Bias** lets the neuron shift its output.
- **Activation** adds non-linearity so the network can learn complex patterns.

2.2 Layers in a Neural Network



Meaning of *Deep*

A neural network becomes **deep** when it has more than one hidden layer. More layers allow the model to learn more abstract patterns, but they also make training more challenging and computationally expensive.

2.3 Common Activation Functions

Activation	Main Idea	Typical Use
ReLU	Keeps positive values, changes negative values to 0	Most hidden layers in modern networks because it is simple and fast
Sigmoid	Converts a value into a score between 0 and 1	Binary classification output such as yes/no prediction
Tanh	Produces values between -1 and 1	Sometimes used in recurrent networks
Softmax	Converts several scores into probabilities that sum to 1	Multi-class classification output such as digit 0 to 9

Easy Memory Trick

Use **ReLU** inside the network, **Sigmoid** for a binary output, and **Softmax** for a multi-class output.

3 How a Neural Network Learns

3.1 Forward Pass

Forward Pass

During the forward pass, the input data moves from the input layer through hidden layers to the output layer. The network produces a prediction such as a class label, a probability, or a numeric value.

3.2 Loss Function

Loss = Prediction Error

The loss function tells the model **how wrong it is**. If the prediction is close to the true answer, the loss is small. If the prediction is far from the true answer, the loss is large.

Typical examples:

- **Binary cross-entropy:** Used for yes/no classification.
- **Categorical cross-entropy:** Used for multi-class classification.
- **Mean squared error:** Used for regression.

3.3 Backpropagation

Backpropagation

Backpropagation is the process of sending the prediction error backward through the network so the weights can be adjusted. In simple words, it answers the question: *Which weights caused the mistake, and how should they change?*

Intuition

Imagine a group project where the final answer is wrong. Backpropagation is like checking each team member's contribution to find who should change their part a little. The network does the same with its weights.

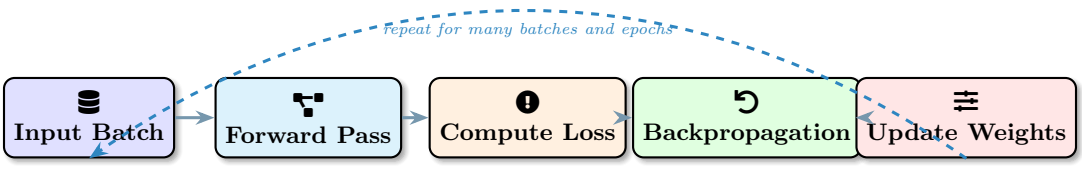
3.4 Gradient Descent, Epochs, and Batches

- **Gradient descent** updates weights to reduce the loss.
- **Learning rate** controls how big each update is.
- **Epoch** means one full pass through the training dataset.
- **Batch** means a smaller chunk of data used in one update step.

Why the Learning Rate Matters

If the learning rate is too small, training is very slow. If it is too large, the model may jump around and fail to learn a stable solution.

3.5 The Training Cycle



3.6 Important Hyperparameters

Hyperparameter	What It Controls
Learning rate	Size of each weight update during training
Batch size	Number of training examples processed at one time
Epochs	Number of full passes through the training data
Number of layers	Depth of the network
Number of neurons	Capacity of each layer to learn patterns
Dropout rate	Fraction of neurons randomly ignored during training to reduce overfitting

4 Key Terminology for Beginners

Term	Meaning
Feature	An input value such as age, pixel intensity, or word index
Weight	A learned value showing how strongly an input affects the output
Bias	An extra learned value that shifts the neuron output
Neuron	A small computational unit that transforms inputs into an output
Layer	A group of neurons at the same stage in the network
Activation function	A rule that transforms a neuron output into a usable signal
Parameter	A value learned during training, such as a weight or bias
Hyperparameter	A value chosen before training, such as learning rate or batch size
Loss	A number showing how wrong the prediction is
Optimizer	The method used to update weights, such as SGD or Adam
Overfitting	When the model memorizes training data and performs poorly on new data
Generalization	The ability to work well on unseen data

5 Preparing Data for Deep Learning

Garbage In, Garbage Out

Even a powerful deep learning model cannot rescue poor data. Clean, relevant, and well-prepared data is still the foundation of good performance.

5.1 How Different Data Types Are Prepared

Data Type	Representation	Common Preparation Steps
Images	Pixel grids	Resize, normalize pixel values, augment with flips or rotations
Text	Tokens or embeddings	Clean text, tokenize, pad sequences, build embeddings
Audio	Waveforms or spectrograms	Trim noise, convert to features, standardize length
Tabular data	Rows and columns	Handle missing values, encode categories, scale numeric features

5.2 Train, Validation, and Test Sets

Data Splitting

Split the dataset into:

- **Training set:** Used to learn the model.
- **Validation set:** Used to tune settings and monitor overfitting.
- **Test set:** Used only at the end for final evaluation.

For many classroom projects, a split such as **70% training, 15% validation, 15% test** is a good starting point.

5.3 Useful Preparation Techniques

- Normalize or standardize numeric values.
- Encode text and categorical variables properly.
- Balance classes when one class is much rarer than the others.
- Use data augmentation for images to create more variety.
- Remove obvious noise, duplicates, and incorrect labels when possible.

A Common Beginner Mistake

Never let the test set influence model design decisions. If you repeatedly tune the model using the test set, your final performance estimate becomes overly optimistic.

6 Common Deep Learning Architectures

Architecture	Best For	Main Idea
MLP (Multilayer Perceptron)	Tabular data, simple classification	Fully connected layers learn patterns from structured inputs
CNN (Convolutional Neural Network)	Images and spatial data	Filters scan local regions and detect visual patterns such as edges and shapes

Architecture	Best For	Main Idea
RNN (Recurrent Neural Network)	Sequential data	The model remembers previous steps while processing the current step
LSTM / GRU	Longer sequences and time series	Improved recurrent design that remembers useful information for longer
Transformer	Text, language, vision, large AI systems	Attention lets the model focus on the most relevant parts of the input
Autoencoder	Compression and anomaly detection	Learns how to encode input into a compact form and reconstruct it

6.1 A CNN Pipeline for Image Tasks

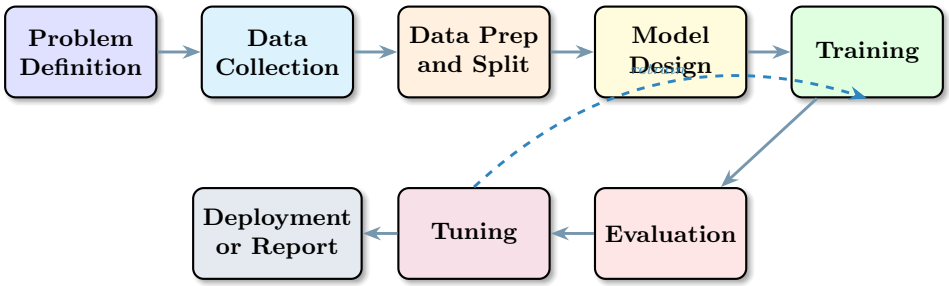
- **CNNs** are strong for image tasks because nearby pixels usually belong together.
- **RNNs and LSTMs** are useful when order matters, such as sentences or time series.
- **Transformers** are now dominant in many language tasks and increasingly in vision tasks.

6.2 Understanding Attention in One Sentence

Attention

Attention helps a model focus on the **most relevant parts of the input** instead of treating every part as equally important. This is one reason transformers work so well in language models and modern AI systems.

7 A Practical Deep Learning Workflow



7.1 Step-by-Step View

1. Define the task clearly: classification, regression, forecasting, or generation.
2. Collect and clean the data.
3. Choose how to represent the data for the model.
4. Select a suitable architecture.

5. Train the model and monitor loss and validation performance.
6. Evaluate using appropriate metrics.
7. Improve the model with tuning, regularization, or better data.
8. Deploy the model or present findings.

8 Easy-to-Understand Examples

8.1 Example 1: Handwritten Digit Recognition

</> How a CNN Reads Digits

Suppose we want to classify images of handwritten digits from 0 to 9.

1. The input is a small grayscale image.
2. Early CNN layers detect edges and simple curves.
3. Later layers combine those patterns into shapes such as loops or corners.
4. The output layer gives probabilities for classes 0, 1, 2, ..., 9.
5. The class with the highest probability becomes the prediction.

This example is popular because it shows how deep learning learns features automatically from images.

8.2 Example 2: Sentiment Analysis of Reviews

</> Positive or Negative?

Input text: *"This mobile app is fast and easy to use."*
Possible workflow:

- Convert words into tokens or embeddings.
- Use an RNN, LSTM, or Transformer to process word order and meaning.
- Predict whether the review is positive, negative, or neutral.

This helps students connect deep learning to real IT applications such as customer feedback analysis and chatbot improvement.

8.3 Example 3: Student Support Prediction

</> A BSIT-Friendly Tabular Example

Imagine a dataset with features such as attendance, laboratory score, quiz average, and assignment completion. A small multilayer perceptron can learn whether a student may need early academic support.
This is useful because it shows that deep learning can also work with **tabular educational data**, not only images and text.

8.4 A Small Keras-Style Example

```python
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(8, activation="relu", input_shape=(4,)),
    Dense(4, activation="relu"),
    Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])

model.fit(X_train, y_train, epochs=20, batch_size=16)
model.evaluate(X_test, y_test)
```

What This Code Shows

The code creates a simple neural network with two hidden layers, chooses an optimizer and loss function, trains on the training set, and then evaluates performance on unseen test data.

9 Evaluation, Overfitting, and Improvement

9.1 Useful Evaluation Metrics

- **Accuracy:** Overall percentage of correct predictions.
- **Precision:** When the model predicts positive, how often is it correct?
- **Recall:** Of all real positives, how many did the model find?
- **F1-score:** Balance between precision and recall.
- **MAE / MSE / RMSE:** Common for regression problems.

9.2 Overfitting vs. Underfitting

Underfitting	Overfitting
Model is too simple	Model memorizes training data
Poor on training data and test data	Very good on training data, weak on new data
Needs more model capacity or better features	Needs regularization, more data, or simpler design

9.3 How to Reduce Overfitting

- Use more training data if available.
- Apply **dropout** so some neurons are randomly ignored during training.
- Use **early stopping** to stop training when validation performance stops improving.
- Add **data augmentation** for image tasks.
- Reduce the model size if it is unnecessarily large.

9.4 Common Problems and Practical Remedies

Problem	Typical Symptom	Practical Remedy
Overfitting	Validation accuracy drops while training accuracy keeps rising	Dropout, early stopping, more data, smaller model
Underfitting	Both training and validation performance stay low	More layers, more epochs, better features, better tuning
Data shortage	Model performance is unstable	Gather more data, augment data, use transfer learning
Class imbalance	Rare cases are missed	Resampling, class weights, better metrics than accuracy alone
High compute cost	Training takes too long	Use smaller model, smaller batch, GPU, or pretrained models

10 Deep Learning vs. Traditional Machine Learning

Aspect	Traditional ML	Deep Learning
Feature engineering	Often manual and important	Often learned automatically
Data requirement	Can work well on smaller datasets	Usually benefits from large datasets
Interpretability	Often easier to explain	Often harder to interpret
Computing need	Usually lower	Usually higher
Best known strengths	Tabular business data, interpretable models	Images, speech, text, complex patterns

Practical Advice for Students

If your dataset is small and highly structured, start with traditional machine learning. If your data is complex, unstructured, and large, deep learning becomes much more attractive.

11 Best Practices Checklist

- Clearly define the target variable and success metric.
- Understand the data before choosing the model.
- Start with a simple baseline before building a deeper network.
- Split data correctly into training, validation, and test sets.
- Monitor both training and validation performance.
- Record hyperparameters and results for each experiment.
- Use regularization methods when overfitting appears.

- Prefer pretrained models when data or time is limited.
- Evaluate the model on unseen data before reporting success.
- Consider fairness, privacy, and bias in the dataset.

12 Mini Case Study for BSIT Data Science

Scenario

A BSIT department wants to identify students who may need extra academic support early in the semester. The available data includes attendance, laboratory activities, quiz averages, assignment completion, and previous subject performance.

Possible workflow:

1. Define the target as *needs support* or *does not need support*.
2. Clean missing records and encode the data.
3. Split the dataset into training, validation, and test sets.
4. Train a small MLP model.
5. Compare it with logistic regression as a baseline.
6. Evaluate using accuracy, precision, and recall.
7. Use the final model carefully as a decision-support tool, not as an automatic final judgment.

Responsible Use

Predictions about students should support teachers and advisers, not replace human judgment. Educational models can be useful, but they must be used ethically and checked for bias.

13 Summary

- Deep learning is a powerful subset of machine learning built from many connected neurons.
- The basic flow is: input data → forward pass → loss → backpropagation → weight update.
- Different architectures suit different data types: MLP for tabular data, CNN for images, RNN/LSTM for sequences, and Transformers for attention-based modeling.
- Good data preparation, proper evaluation, and regularization are essential.
- For BSIT Data Science students, deep learning is most useful when the problem involves complex patterns in large or unstructured data.

14 Review Questions

1. What is the difference between artificial intelligence, machine learning, and deep learning?
2. What does a neuron do inside a neural network?

3. Why do neural networks need activation functions?
4. In simple words, what is the purpose of backpropagation?
5. What is the difference between a parameter and a hyperparameter?
6. Why do we use separate validation and test sets?
7. Which architecture would you choose for image classification, and why?
8. What is overfitting, and how can it be reduced?
9. Why might a traditional machine learning model still be better than deep learning in some projects?
10. How can deep learning be applied responsibly in an educational setting?